Client VS Server Side Rendering

Server-Side Rendering (SSR) in Next.js

A technique where initial HTML is generated on the server for every request, improving SEO and performance.

How Next.js Implements SSR

- **The Request:** Server receives the user request.
- **Data Fetching:** getServerSideProps executes on the server.
- **HTML Generation:** Data is passed to components and rendered to HTML.
- **Sending Response:** Fully rendered HTML sent to browser.
- **Hydration:** React takes over to make the page interactive.

Key Benefits

• **Improved SEO:** Search engines index fully rendered HTML easily.

• **Faster Content:** Better perceived performance and UX.

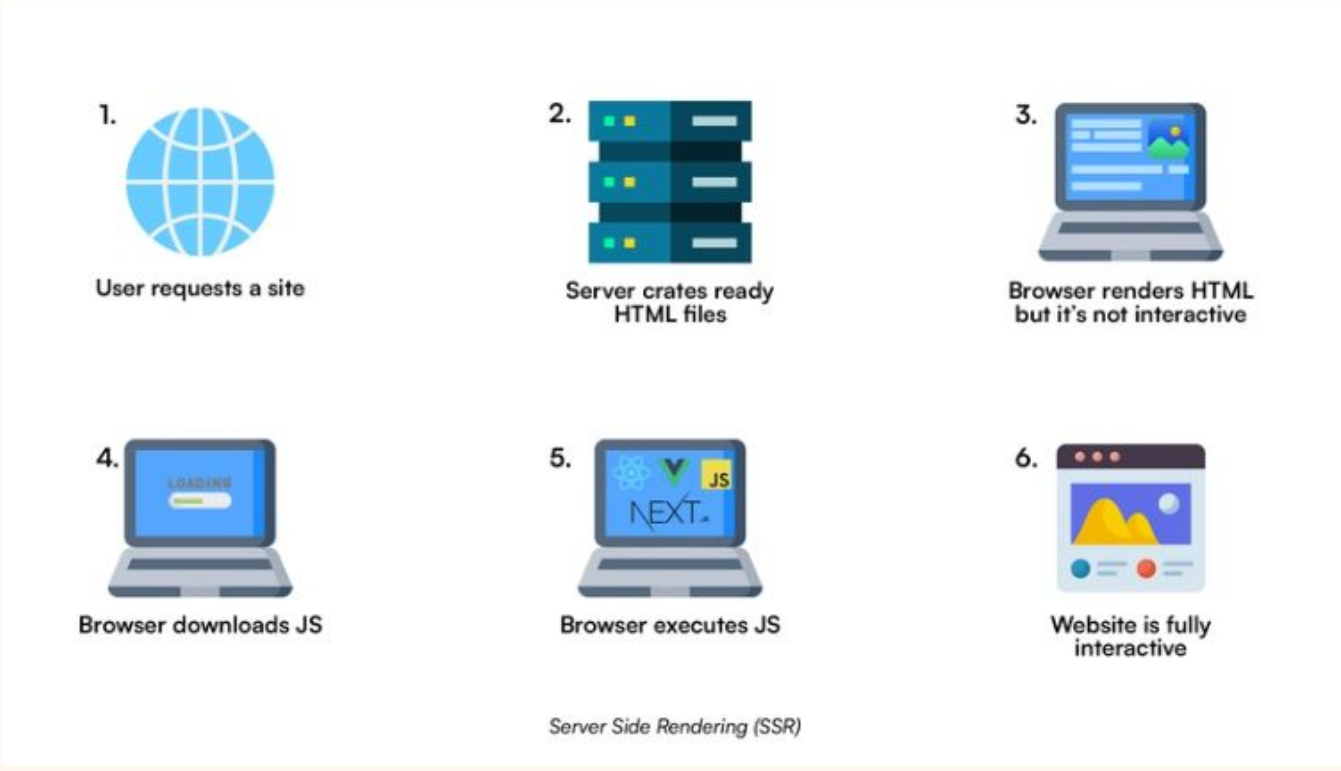
• **Up-to-date:** Data is fresh on every request.

When to Use SSR

• Dynamic, fresh content (tickers, dashboards).

• Request-dependent data (cookies, URL params).

Server-Side Rendering (SSR) in Next.js



Server Side Rendering (SSR)

Client-Side Rendering (CSR) in Next.js

A React approach where rendering and data fetching are handled primarily by the browser after the initial page load.

How Next.js Implements CSR

- **Initial Render:** Browser receives minimal HTML/shell.
- **Bundles:** Browser downloads/executes JS.
- **Data Fetching:** useEffect (or SWR/React Query) triggers API request.
- **Final Render:** State updates and React renders data-rich content.

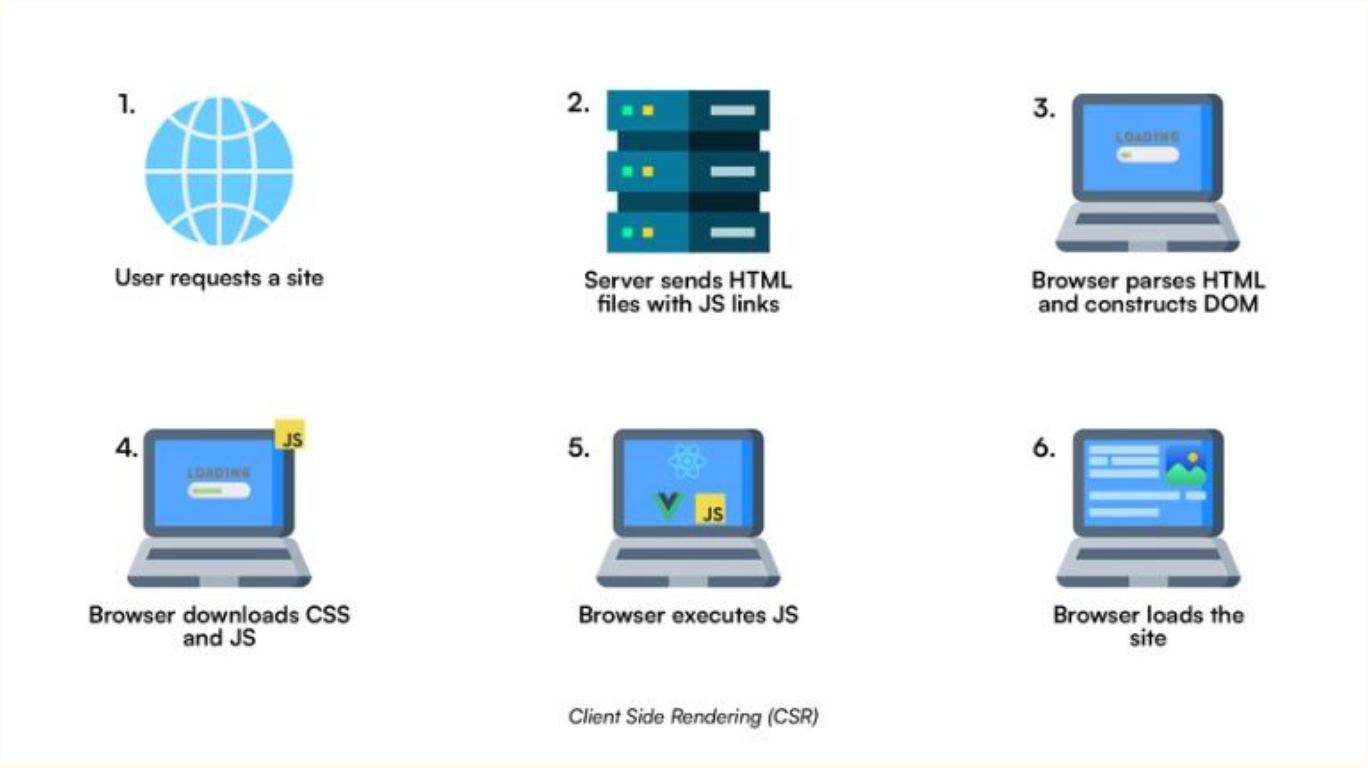
Key Benefits

• **Fast Shell:** Quick initial load of application shell.
• **Reduced Server Load:** Server only serves static files.
• **UX:** Fast subsequent client-side navigation.

Drawbacks & When to Use

• **Drawbacks:** Slower time to content; SEO challenges.
• **Use Cases:** User-specific/auth content; non-SEO pages.

Client-Side Rendering (CSR) in Next.js



Client Side Rendering (CSR)

SSR vs. CSR: Choosing the Right Strategy

Deciding between SSR and CSR depends on your specific requirements for data freshness, performance, and SEO.

When to Use SSR

- **SEO is Critical:** Search engines easily index fully rendered HTML delivered from the server.
- **Dynamic Content:** Necessary when content must be fresh on every request (e.g., dashboards).
- **Request-Dependent Data:** If data fetching relies on cookies or URL parameters.
- **Perceived Performance:** Delivers a complete page quickly for a better initial UX.

When to Use CSR

- **User-Specific Content:** Ideal for authenticated sections like profile settings.
- **Non-SEO Pages:** Use when search engine visibility is not a priority for the page.
- **Fast Initial Shell:** Offers a quick initial load of the application's structure.
- **Navigation Speed:** Fast subsequent client-side navigation between pages.
- **Reduced Server Load:** Offloads rendering logic to the client's browser.